

ArtConnect Application

Database 2: Security and Automation

Project

Database project - Step 1

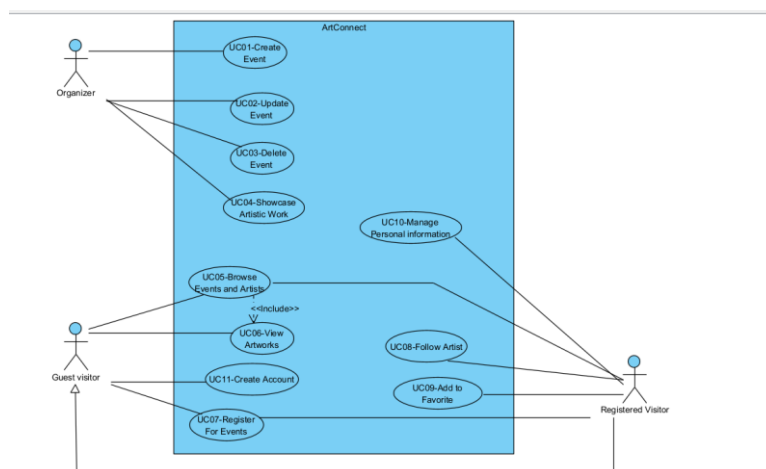
Main features seen in the interface:

- Discover: allows the user to see the next exhibitions and workshops
- Artists: list of artists, you can search for one and sort them (by name, city, email, birth year)
- Artworks: list of artworks, you can also sort them (notably by artist, title, price, status and type)
- Galleries: list of galleries with their addresses and ratings
- Exhibitions: list of exhibitions, we can sort them by title, start date, gallery and theme
- Workshops: list of workshops, can be sorted by title, instructor, date, price and level
- Community: list of communities, which can be sorted by name, email and city

Identify at least two roles or users profile:

- 1- Organizer:
 - 1.1- Create events (Concerts, workshops,ect..)
 - 1.2- Update (modify) event information
 - 1.3- Delete events
 - 1.4- Showcase their artistic work
- 2- Visitor:
 - 2.1- Browse the events and artists
 - 2.2- View artworks
 - 2.3- Register for events

Use Case Diagram:

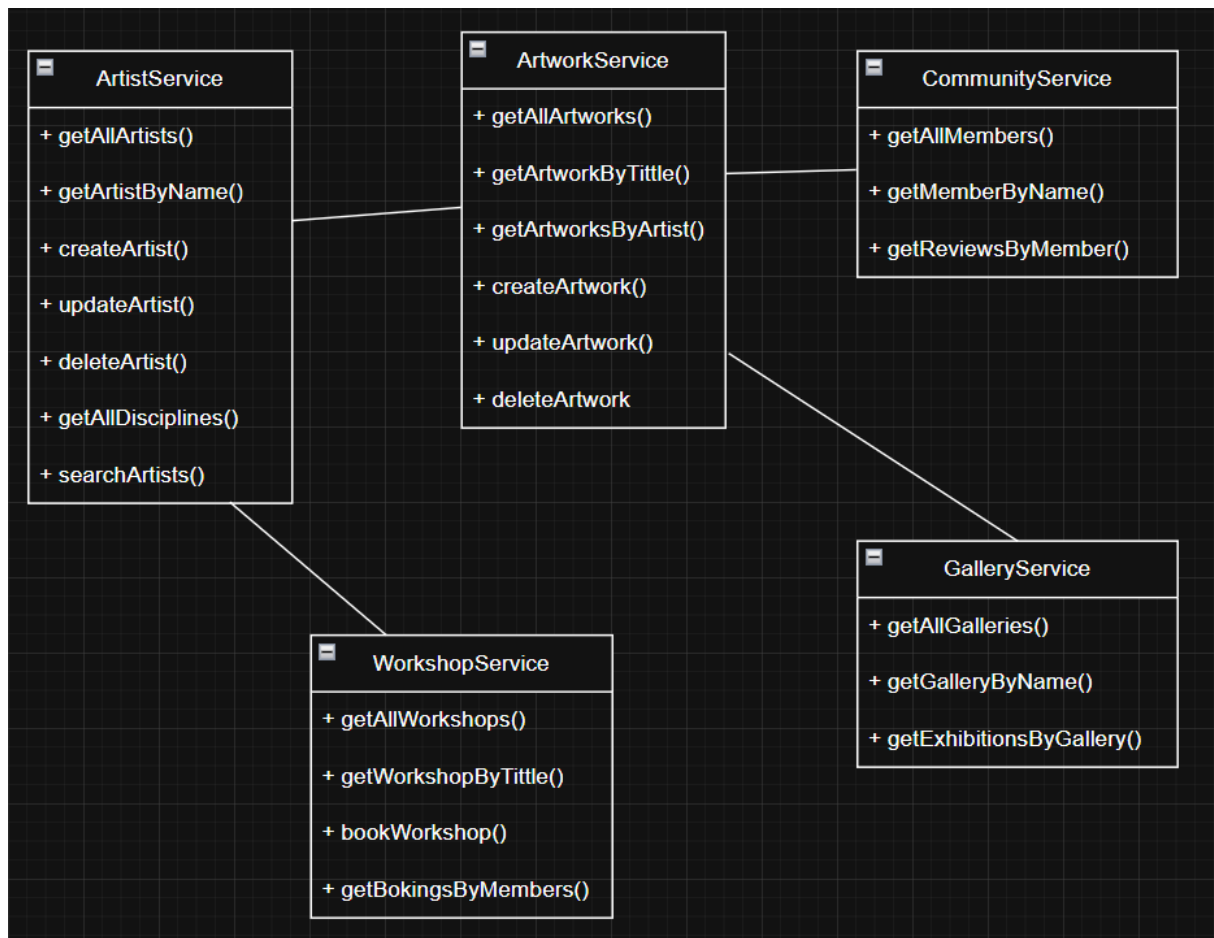


Possible functionalities that could be added

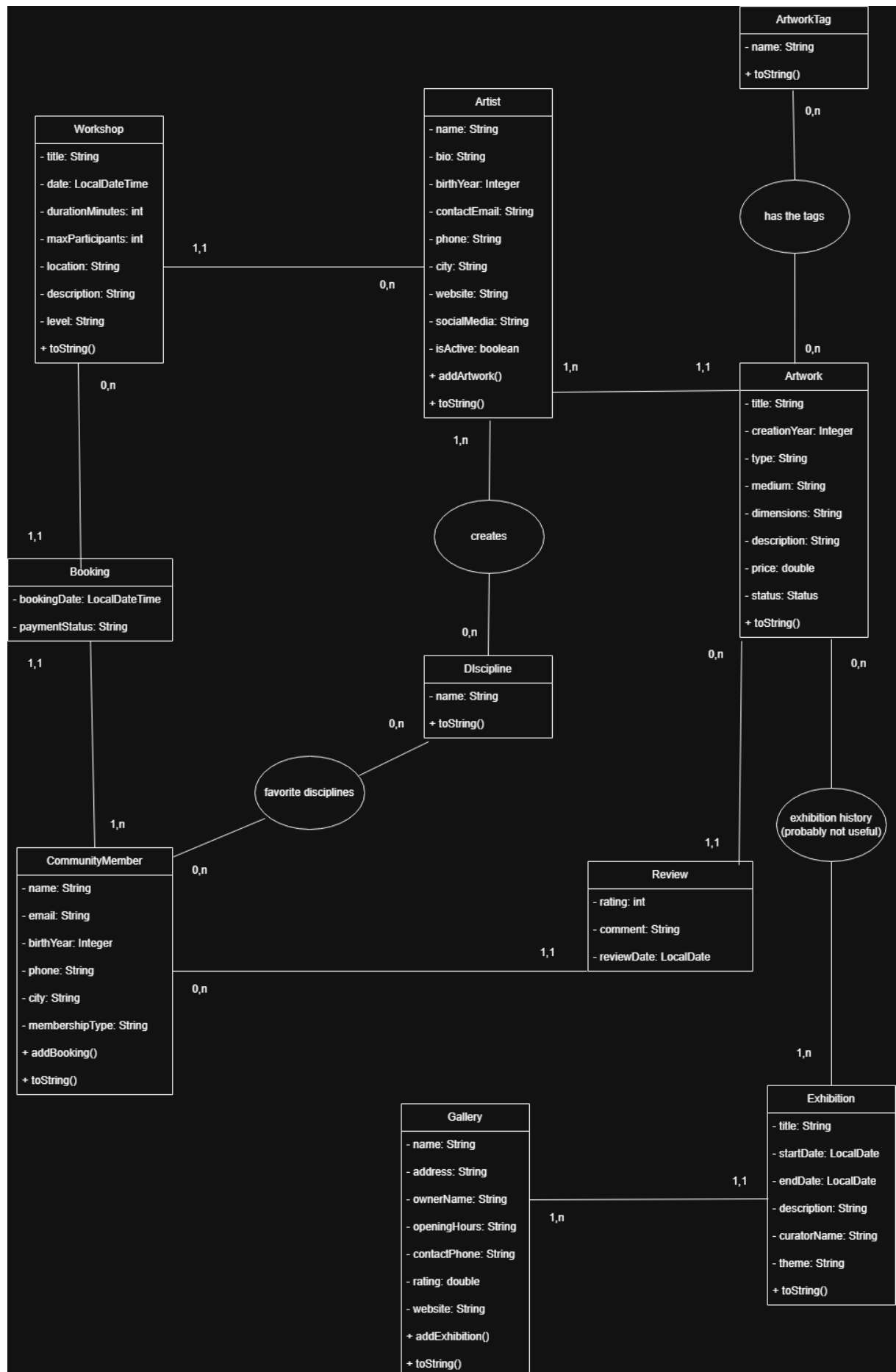
- UC12-Evaluate an artwork
- UC13-Send a message
- UC14-Add friends
- UC15-Create artist account
- UC16-Create art categories
- UC17-Evaluate galleries

Class diagrams:

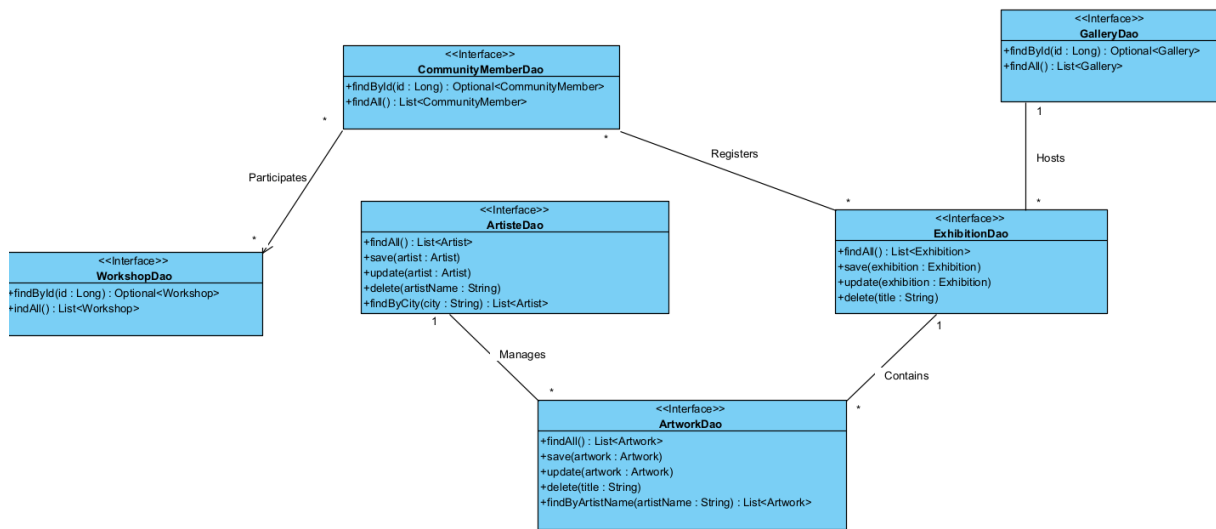
- Services class diagram



- Model class diagram



- DAOs



Database Project – Step 2

1)

Artists: ArtistID, Aname, Acity, Aemail, AbirthYear, Abio, Aphone, Awebsite, AsocialMedia, isActive, #Adiscipline (refers to Name in Discipline)

Artworks: ArtworkID, AWtitle, AWtype, AWprice, AWstatus, AWcreationYear, AWmedium, AWdimensions, AWdescription, #AWartist, #AWtag (refers to AWtname)

ArtworkTag: AWtname

Booking: BookingID, BookingDate, PaymentStatus, #Member (refers to MemberID), #Workshop (refers to Title in Workshops)

Exhibitions: ExhibitionID, EXtitle, EXstartDate, EXtheme, EXendDate, EXdescription, EXcuratorName, #EXartworks (refers to ArtworkID), #Gallery (refers to GalleryID)

Workshops: WorkshopID, WStitle, WSdate, WSprice, WSlevel, DurationMinutes, MaxParticipants, WSlocation, WSdescription, #WSinstructor (refers to ArtistID)

CommunityMember: MemberID, CMname, CMemail, CMcity, CMbirthYear, CMphone, MembershipType, #CMbookings (refers to BookingID), #CMreviews (refers to ReviewID), #FavoriteDiscipline (refers to Name in Discipline)

Gallery: GalleryID, Gname, Gaddress, Grating, GownerName, GopeningHours, GcontactPhone, Gwebsite, #Gexhibition (refers to ExhibitionID)

Discipline: Dname

Review: ReviewID, Rating, Comment, ReviewDate, #Reviewer (refers to MemberID), #Artwork (refers to ArtworkID)

Functional dependencies:

ArtistID -> Aname, Acity, Aemail, AbirthYear, Abio, Aphone, Awebsite, AsocialMedia, isActive, Dname

ArtworkID -> AWTitle, AWType, AWPrice, AWStatus, AWCreationYear, AWMedium, AWDimensions, AWDescription, ArtistID, AWName (ArtworkTag)

BookingID -> BookingDate, PaymentStatus, Member (CommunityMember), Artwork (ArtworkID)

ExhibitionID -> EXtitle, EXstartDate, EXtheme, EXendDate, EXdescription, EXcuratorName, ArtworkID (Artworks)

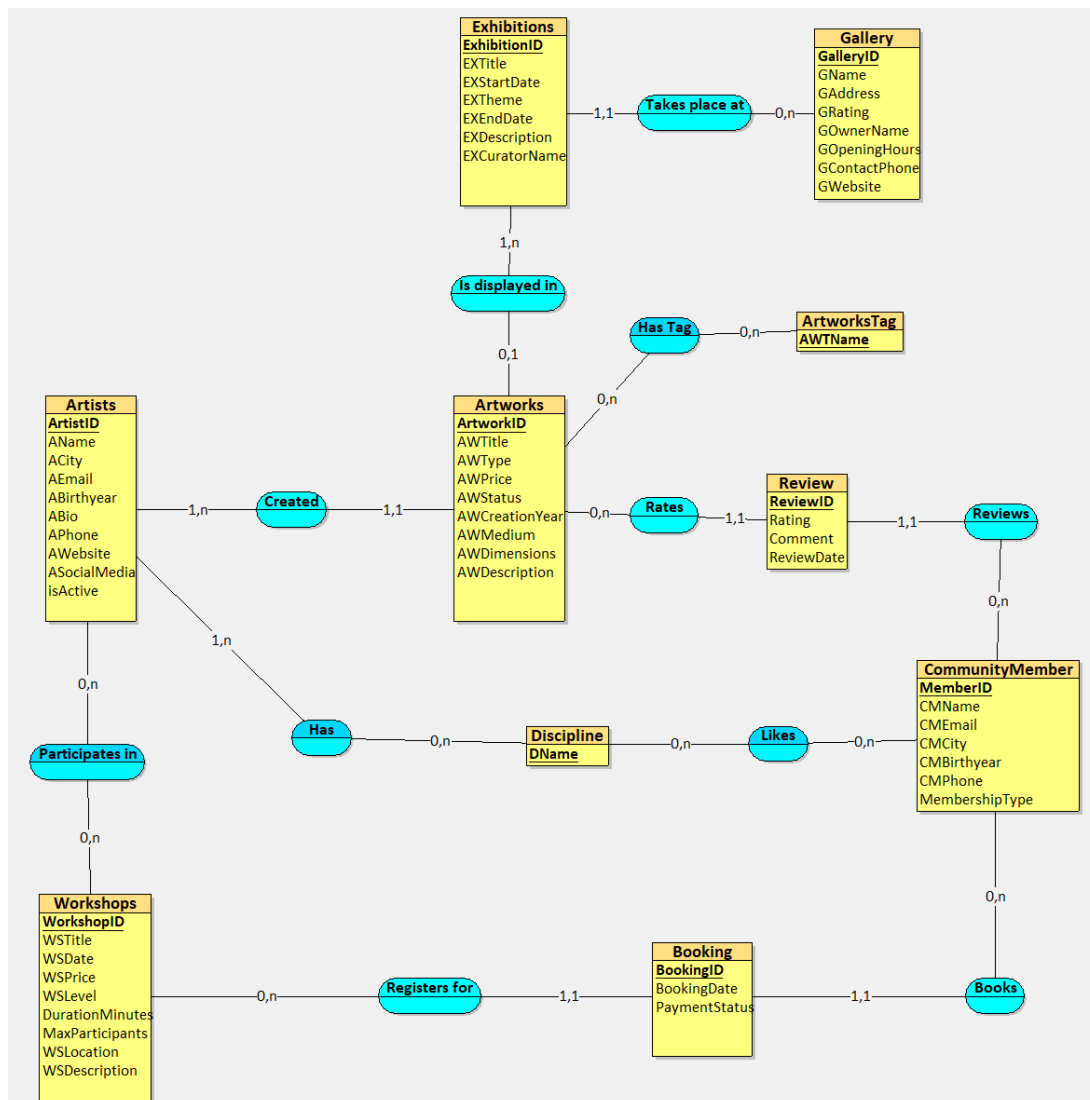
WorkshopID -> WSTitle, WSDate, WSPrice, WSLevel, DurationMinutes, MaxParticipants, WSLocation, WSDescription

MemberID -> CMname, CMemail, CMcity, CMbirthYear, CMphone, MembershipType, #CMbookings (refers to BookingID), #CMreviews (refers to ReviewID), #FavoriteDiscipline (refers to Name in Discipline)

GalleryID -> GalleryID, Gname, Gaddress, Grating, GownerName, GopeningHours, GcontactPhone, Gwebsite, #Gexhibition (refers to ExhibitionID)

ReviewID -> Rating, Comment, ReviewDate, Reviewer (MemberID), Artwork (ArtworkID)

2)



3)

Artists = (ArtistID INT, Aname VARCHAR(50), Acity VARCHAR(50), Aemail VARCHAR(50), AbirthYear DATE, Abio VARCHAR(200), Aphone VARCHAR(20), Awebsite VARCHAR(100), AsocialMedia VARCHAR(50), isActive BOOLEAN);

ArtworksTag = (AWTName VARCHAR(25));

Workshops = (WorkshopID INT, WStitle VARCHAR(100), WSdate DATE, WSprice DECIMAL(3,2), WSlevel VARCHAR(12), DurationMinutes INT, MaxParticipants INT, WSlocation VARCHAR(100), WSdescription VARCHAR(200));

CommunityMember = (MemberID INT, CMname VARCHAR(50), CMemail VARCHAR(50), CMcity VARCHAR(50), CMbirthYear DATE, CMphone VARCHAR(20), MembershipType VARCHAR(7));

Gallery = (GalleryID INT, Gname VARCHAR(50), Gaddress VARCHAR(50), Grating DECIMAL(1,1), GownerName VARCHAR(50), GopeningHours VARCHAR(100), GcontactPhone VARCHAR(20), Gwebsite VARCHAR(100));

Discipline = (DName VARCHAR(50));

Booking = (BookingID INT, BookingDate DATE, PaymentStatus VARCHAR(9), #WorkshopID, #MemberID);

Exhibitions = (ExhibitionID INT, EXtitle VARCHAR(100), EXstartDate DATE, EXtheme VARCHAR(50), EXendDate DATE, EXdescription VARCHAR(200), EXcuratorName VARCHAR(50), #GalleryID);

Artworks = (ArtworkID INT, AWtitle VARCHAR(100), AWtype VARCHAR(25), AWprice DECIMAL(9,2), AWstatus VARCHAR(9), AWcreationYear INT, AWmedium VARCHAR(20), AWdimensions VARCHAR(20), AWdescription VARCHAR(200), #ExhibitionID, #ArtistID);

Review = (ReviewID INT, Rating DECIMAL(1,1), Comment VARCHAR(500), ReviewDate DATE, #ArtworkID, #MemberID);

Has = (#ArtistID, #DName);

Likes = (#MemberID, #DName);

Has_Tag = (#ArtworkID, #AWTName);

Participates_in = (#ArtistID, #WorkshopID);

Right off the bat, we already are in 1NF since all of the attributes are atomic. In addition, every single attribute depends on their respective primary keys so this is also in 2NF and this can also be checked thanks to the functional dependencies we have written somewhere above. Finally, also thanks to those dependencies we know that all of the attributes don't depend on non-key attributes so this fits 3NF. Because we have carefully crafted these tables we didn't have to change them to be in 3NF.

Database Project – Step 3

1. Claude AI prompt:

In « step 3 - prompt.txt »

2. Views (in « step 3 - views.txt »)

- Hide sensitive artist contact info:

```
CREATE VIEW vw_artist_public AS
SELECT
  Aname,
  Acity,
  Abio,
  Awebsite,
  AsocialMedia,
  isActive
FROM Artists;
```

Objective: Hides private fields (Aemail, Aphone, AbirthYear, ArtistID) from public-facing queries. Only expose what a visitor of the platform needs to see.

- Member profile without personal details:

```
CREATE VIEW vw_member_public AS
SELECT
  MemberID,
  CMname,
  CMcity,
  MembershipType
FROM CommunityMember;
```

Objective: Hides sensitive data (CMemail, CMphone, CMbirthYear) from users who only need to reference a member by name or membership type (e.g., for booking lists).

- Available artworks with artist name:

```

CREATE VIEW vw_available_artworks AS
SELECT
    aw.ArtworkID,
    aw.AWtitle,
    aw.AWtype,
    aw.AWprice,
    aw.AWmedium,
    aw.AWdimensions,
    ar.Aname    AS ArtistName,
    ar.Acity    AS ArtistCity
FROM Artworks aw
JOIN Artists ar ON aw.ArtistID = ar.ArtistID
WHERE aw.AWstatus = 'Available';

```

Objective: Simplifies any query that needs to display artworks for sale — avoids repeating the JOIN and WHERE filter every time. Useful for a storefront or catalogue page.

2. Indexes (in « step 3 - indexes.txt »)

- Index on Artworks.AWstatus

```
CREATE INDEX idx_artworks_status ON Artworks(AWstatus);
```

Why? Filtering artworks by status ('Available', 'Sold', etc.) is a good choice of index. Without an index, every query scans the entire table. Since status has low cardinality, this index is lightweight but highly effective for frequent filters.

- Index on Booking.WorkshopID

```
CREATE INDEX idx_booking_workshopid ON Booking(WorkshopID);
```

Why? Without this index, every booking insertion triggers a full table scan.

- Index on Review.ArtworkID

```
CREATE INDEX idx_review_artworkid ON Review(ArtworkID);
```

Why? Since ArtworkID is a foreign key in Review but not the primary key, it won't be automatically indexed so this index must be added explicitly.

3. Triggers (in « step 3 - triggers.txt »)

- Checking workshop date (workshop booking date must not be after workshop date)

```

DELIMITER //
CREATE TRIGGER check_booking_date BEFORE INSERT ON Booking FOR EACH ROW
BEGIN
    DECLARE ws_date DATE;
    SELECT WSdate INTO ws_date FROM Workshops WHERE WorkshopID =
NEW.WorkshopID;
    IF NEW.BookingDate > ws_date THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Booking date cannot be after the workshop date.';
    END IF;
END;
//
DELIMITER ;

```

- Checking workshop capacity (we want to prevent booking a workshop over capacity)

```

DELIMITER //
CREATE TRIGGER trg_check_capacity BEFORE INSERT ON Booking FOR EACH ROW
BEGIN
    DECLARE current_count INT;
    DECLARE max_cap INT;

    SELECT COUNT(*) INTO current_count FROM Booking WHERE WorkshopID =
NEW.WorkshopID;

    SELECT MaxParticipants INTO max_cap FROM Workshops WHERE WorkshopID =
NEW.WorkshopID;

    IF current_count >= max_cap THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Workshop is fully booked. No seats available.';
    END IF;

```

```
END;
```

```
//
```

```
DELIMITER ;
```

- Logging changes (log any update to an exhibition)

```
-- Audit log table
```

```
CREATE TABLE ExhibitionAuditLog (
  LogID    INT AUTO_INCREMENT PRIMARY KEY,
  ExhibitionID INT,
  ChangedAt DATETIME DEFAULT NOW(),
  ChangedField VARCHAR(50),
  OldValue  VARCHAR(200),
  NewValue  VARCHAR(200)
);
```

```
DELIMITER //
```

```
CREATE TRIGGER trg_audit_exhibition_update
AFTER UPDATE ON Exhibitions
FOR EACH ROW
BEGIN
  IF OLD.EXtitle <> NEW.EXtitle THEN
    INSERT INTO ExhibitionAuditLog (ExhibitionID, ChangedField, OldValue, NewValue)
    VALUES (OLD.ExhibitionID, 'EXtitle', OLD.EXtitle, NEW.EXtitle);
  END IF;
  IF OLD.EXstartDate <> NEW.EXstartDate THEN
    INSERT INTO ExhibitionAuditLog (ExhibitionID, ChangedField, OldValue, NewValue)
    VALUES (OLD.ExhibitionID, 'EXstartDate', OLD.EXstartDate, NEW.EXstartDate);
  END IF;
  IF OLD.EXendDate <> NEW.EXendDate THEN
    INSERT INTO ExhibitionAuditLog (ExhibitionID, ChangedField, OldValue, NewValue)
    VALUES (OLD.ExhibitionID, 'EXendDate', OLD.EXendDate, NEW.EXendDate);
  END IF;
END;
//
DELIMITER ;
```

3. Stored procedures

- Create a Workshop and automatically register its instructor (artist)

```

DELIMITER //
CREATE PROCEDURE sp_create_workshop_and_register_artist(
    IN p_title    VARCHAR(100),
    IN p_date     DATE,
    IN p_price    DECIMAL(9,2),
    IN p_level    VARCHAR(12),
    IN p_duration INT,
    IN p_maxPart  INT,
    IN p_location VARCHAR(100),
    IN p_description VARCHAR(200),
    IN p_artistID INT
)
BEGIN
    DECLARE new_workshop_id INT;

    -- Create the workshop
    INSERT INTO Workshops (WStitle, WSdate, WSprice, WSlevel, DurationMinutes,
        MaxParticipants, WSlocation, WSdescription)
    VALUES (p_title, p_date, p_price, p_level, p_duration,
        p_maxPart, p_location, p_description);

    SET new_workshop_id = LAST_INSERT_ID();

    -- Automatically register the instructor (artist) as a participant
    INSERT INTO Participates_in (ArtistID, WorkshopID)
    VALUES (p_artistID, new_workshop_id);

    SELECT new_workshop_id AS CreatedWorkshopID;
END;
//
DELIMITER ;

```

- Function – Return the number of participants for a given workshop

```

DELIMITER //
CREATE FUNCTION fn_count_participants(p_workshopID INT)
RETURNS INT
DETERMINISTIC
BEGIN

```

```

DECLARE participant_count INT;
SELECT COUNT(*) INTO participant_count
FROM Booking
WHERE WorkshopID = p_workshopID;
RETURN participant_count;
END;
//
DELIMITER ;

```

4. Transactions

Scenario: Member purchases an artwork and books a related workshop

This transaction covers a realistic real-world flow: a member buys an artwork and simultaneously books a workshop taught by that artwork's artist. All steps must succeed together or the entire operation is rolled back.

The steps that must be atomic are:

Verify the artwork is available and the workshop has capacity

Mark the artwork as 'Sold'

Create a booking for the workshop

Create a review placeholder entry (optional but linked)

If anything fails → roll back everything

```
DELIMITER //
```

```

CREATE PROCEDURE sp_purchase_and_book(
  IN p_memberID INT,
  IN p_artworkID INT,
  IN p_workshopID INT,
  IN p_bookingDate DATE
)

```

```
BEGIN
```

```

  -- — Declare variables —————
  DECLARE v_status    VARCHAR(9);
  DECLARE v_currentCount INT;
  DECLARE v_maxPart   INT;
  DECLARE v_wsDate    DATE;

```

```

DECLARE v_artistID    INT;
DECLARE v_alreadyBooked INT;

-- Declare EXIT HANDLER: if anything goes wrong, rollback everything
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Transaction failed. All changes have been rolled back.';
END;

-- ——— Start transaction —————
START TRANSACTION;

-- ——— Step 1: Check artwork is available —————
SELECT AWstatus, ArtistID
INTO v_status, v_artistID
FROM Artworks
WHERE ArtworkID = p_artworkID
FOR UPDATE; -- Lock the row to prevent concurrent purchases

IF v_status <> 'Available' THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Artwork is not available for purchase.';
END IF;

-- ——— Step 2: Check workshop capacity —————
SELECT WSdate, MaxParticipants
INTO v_wsDate, v_maxPart
FROM Workshops
WHERE WorkshopID = p_workshopID
FOR UPDATE; -- Lock to prevent race condition on capacity

SELECT COUNT(*)
INTO v_currentCount
FROM Booking
WHERE WorkshopID = p_workshopID;

IF v_currentCount >= v_maxPart THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Workshop is fully booked.';

```

```

END IF;

-- — Step 3: Check booking date is valid —————
IF p_bookingDate > v_wsDate THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Booking date cannot be after the workshop date.';
END IF;

-- — Step 4: Check member hasn't already booked this workshop —————
SELECT COUNT(*)
INTO v_alreadyBooked
FROM Booking
WHERE WorkshopID = p_workshopID
    AND MemberID = p_memberID;

IF v_alreadyBooked > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Member is already booked for this workshop.';
END IF;

-- — Step 5: Mark artwork as Sold —————
UPDATE Artworks
SET AWstatus = 'Sold'
WHERE ArtworkID = p_artworkID;

-- — Step 6: Create the workshop booking —————
INSERT INTO Booking (BookingDate, PaymentStatus, WorkshopID, MemberID)
VALUES (p_bookingDate, 'Pending', p_workshopID, p_memberID);

-- — Step 7: Verify the workshop's instructor is the artwork artist
--           and register them as participant if not already registered
IF NOT EXISTS (
    SELECT 1 FROM Participates_in
    WHERE ArtistID = v_artistID
        AND WorkshopID = p_workshopID
) THEN
    INSERT INTO Participates_in (ArtistID, WorkshopID)
    VALUES (v_artistID, p_workshopID);
END IF;

-- — All good: commit —————

```



```
COMMIT;

SELECT 'Transaction successful.' AS Result,
       p_artworkID      AS PurchasedArtwork,
       p_workshopID     AS BookedWorkshop,
       p_memberID       AS MemberID;

END;
//
DELIMITER ;
```

Usage:

```
CALL sp_purchase_and_book(
  7,      -- MemberID
  42,     -- ArtworkID
  3,      -- WorkshopID
  '2025-05-20' -- BookingDate
);
```

Database Project – Step 4

Brief Description of the architecture:

The key idea is that each layer only talks to the one directly below it:

UI → Service → DAO → Database

UI (User Interface) The screens the user sees and interacts with. It just displays data and captures actions (clicks, form inputs). It doesn't know anything about the database.

Service Layer The "brain" of the app. It contains the business rules, for example "is this artwork available before buying it?" It receives requests from the UI and decides what to do.

DAO (Data Access Object) The only layer that talks to the database. It contains all the SQL queries (SELECT, INSERT, UPDATE, DELETE). Nothing else in the app writes SQL.

Database (MySQL) Where all the data actually lives permanently. Without this, everything resets when you close the app.

And here as we can see below the database was linked successfully by looking at both the SQL inserts and the UI that we have:

Name	City	Email	Birth Year
Elena Moreau	Paris	elena@moreau-art.fr	1985
Karim Osei	Lyon	karim@oseistudio.fr	1978
Yuki Tanaka	Bordeaux	yuki@tanaka-photo.fr	1990
Sophie Renard	Marseille	sophie@renard-prints.fr	1982
Marco Benedetti	Nice	marco@benedetti-art.it	1975
Ines Dupuis	Toulouse	ines@dupuis-digital.fr	1995

```
INSERT INTO CommunityMember
(MemberID, CMName, CMEmail, CMCity, CMBirthyear, CMPhone, MembershipType)
VALUES
(1, 'Alice Fontaine', 'alice@mail.fr', 'Paris', '1988-04-02', '+33 6 01 11 22 33', 'Premium'),
(2, 'Bruno Sauvage', 'bruno@mail.fr', 'Lyon', '1975-08-19', '+33 6 02 11 22 33', 'Standard'),
(3, 'Céline Morel', 'celine@mail.fr', 'Bordeaux', '1992-12-25', '+33 6 03 11 22 33', 'Premium'),
(4, 'David Kante', 'david@mail.fr', 'Marseille', '1983-03-07', '+33 6 04 11 22 33', 'Basic'),
(5, 'Emma Leclaire', 'emma@mail.fr', 'Nice', '1997-06-14', '+33 6 05 11 22 33', 'Standard'),
(6, 'Félix Garnier', 'felix@mail.fr', 'Toulouse', '1969-11-30', '+33 6 06 11 22 33', 'Premium'),
(7, 'Gaëlle Perrin', 'gaelle@mail.fr', 'Paris', '2001-01-08', '+33 6 07 11 22 33', 'Basic'),
(8, 'Hugo Descartes', 'hugo@mail.fr', 'Lyon', '1986-09-21', '+33 6 08 11 22 33', 'Standard');
```

Database Project – Step 5

Presentation slides link: <https://canva.link/db6d8pa373dbts0>

Test scenario:

Member purchases an artwork and books a related workshop

This transaction covers a realistic real-world flow: a member buys an artwork and simultaneously books a workshop taught by that artwork's artist. All steps must succeed together or the entire operation is rolled back.

The steps that must be atomic are:

Verify the artwork is available and the workshop has capacity

Mark the artwork as 'Sold'

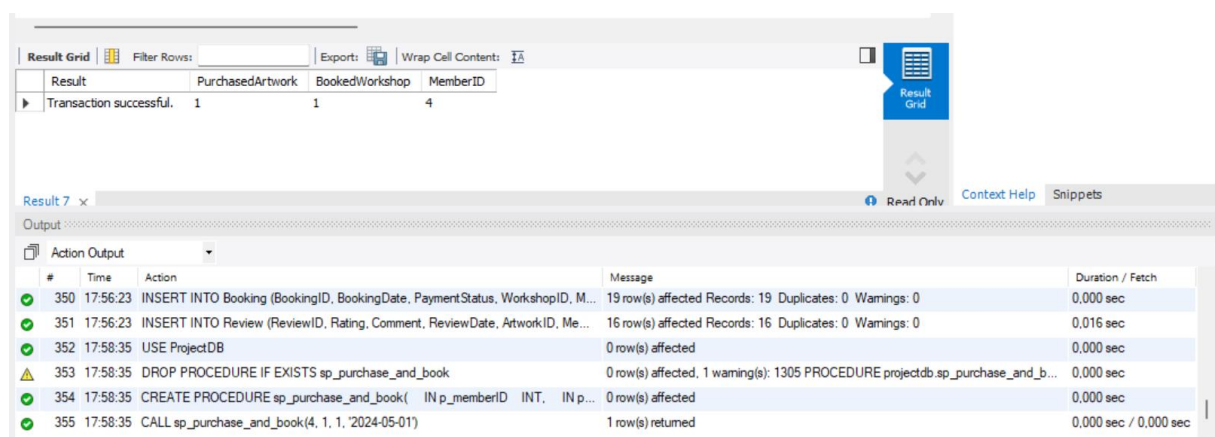
Create a booking for the workshop

Create a review placeholder entry (optional but linked)

If anything fails → roll back everything

So we simply need to test `sp_purchase_and_book` (**video available on github**).

This is its output on mysql:



#	Time	Action	Message	Duration / Fetch
350	17:56:23	INSERT INTO Booking (BookingID, BookingDate, PaymentStatus, WorkshopID, M...	19 row(s) affected Records: 19 Duplicates: 0 Warnings: 0	0,000 sec
351	17:56:23	INSERT INTO Review (ReviewID, Rating, Comment, ReviewDate, ArtworkID, Me...	16 row(s) affected Records: 16 Duplicates: 0 Warnings: 0	0,016 sec
352	17:58:35	USE ProjectDB	0 row(s) affected	0,000 sec
353	17:58:35	DROP PROCEDURE IF EXISTS sp_purchase_and_book	0 row(s) affected, 1 warning(s): 1305 PROCEDURE projectdb.sp_purchase_and_b...	0,000 sec
354	17:58:35	CREATE PROCEDURE sp_purchase_and_book(IN p_memberID INT, IN p...	0 row(s) affected	0,000 sec
355	17:58:35	CALL sp_purchase_and_book(4, 1, 1, '2024-05-01')	1 row(s) returned	0,000 sec / 0,000 sec